



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 05

NOMBRE COMPLETO: Antonio Manuel Zamora Ayala

N° de Cuenta: 320152896

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 26 de septiembre de 2026

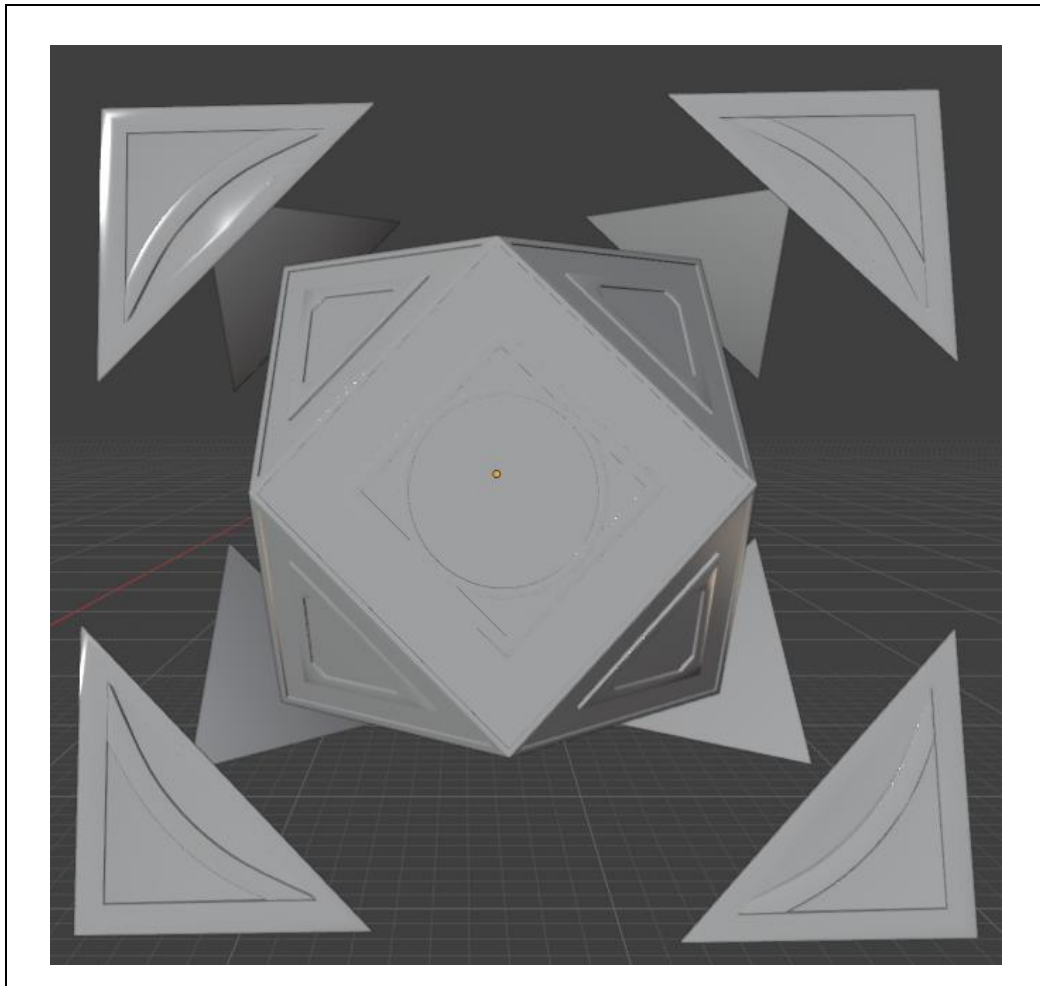
CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1. Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1.1 Importar el modelo del Holocron con jerarquía y teclado de forma independiente poder girar cada esquina alrededor del centro del Holocron.

Primero importé y separé en *Blender* todos los elementos de interés del modelo *Holocron.obj* (8 pirámides y la parte central del holocron) y *Satellite_320152896.fbx* (2 paneles solares, una antena y al base del satélite).



Posteriormente exporté cada elemento por separado a la carpeta *Models* y una vez hecho esto, en *P05_320152896.cpp* modifiqué los bloques de código siguientes:

- Lista de modelos a importar:

```
54 //Holocron
55 Model centroHolocron;
56 Model piramide1;
57 Model piramide2;
58 Model piramide3;
59 Model piramide4;
60 Model piramide5;
61 Model piramide6;
62 Model piramide7;
63 Model piramide8;
```

- Carga de modelos:

```
186 //Holocron
187 centroHolocron = Model();
188 centroHolocron.LoadModel("Models/centroHolocron.fbx");
189
190 piramide1 = Model();
191 piramide1.LoadModel("Models/piramide1.fbx");
192
193 piramide2 = Model();
194 piramide2.LoadModel("Models/piramide2.fbx");
195
196 piramide3 = Model();
197 piramide3.LoadModel("Models/piramide3.fbx");
198
199 piramide4 = Model();
200 piramide4.LoadModel("Models/piramide4.fbx");
201
202 piramide5 = Model();
203 piramide5.LoadModel("Models/piramide5.fbx");
204
205 piramide6 = Model();
206 piramide6.LoadModel("Models/piramide6.fbx");
207
208 piramide7 = Model();
209 piramide7.LoadModel("Models/piramide7.fbx");
210
211 piramide8 = Model();
212 piramide8.LoadModel("Models/piramide8.fbx");
```

➤ Dibujo de los objetos:

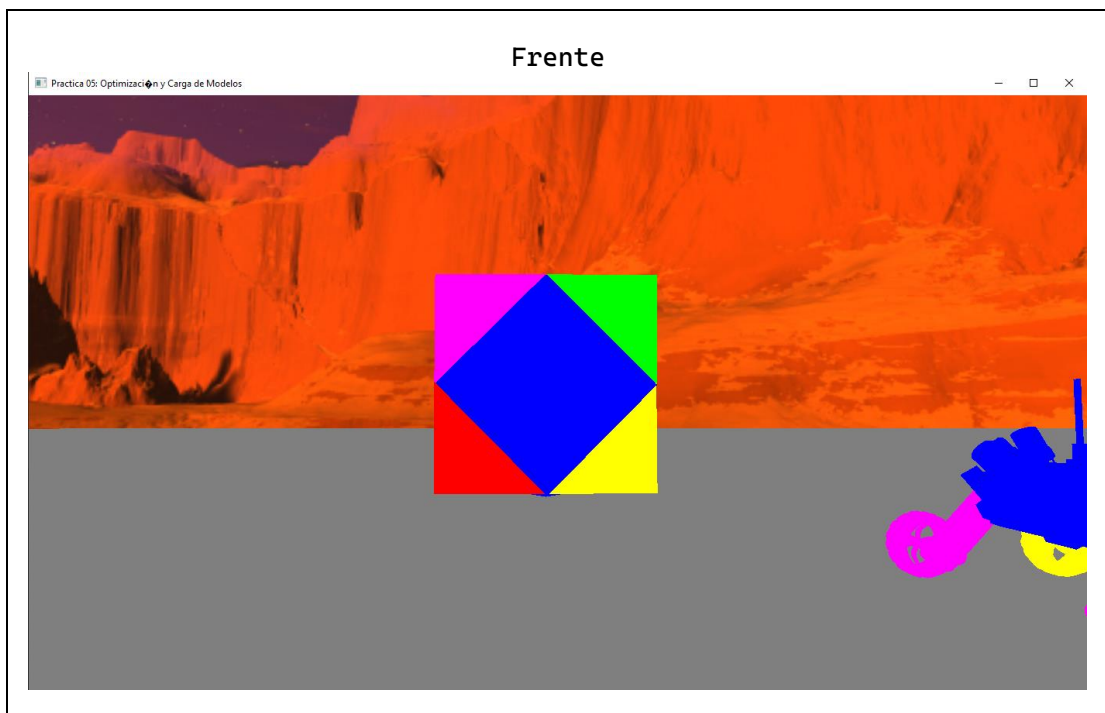
```
245 glm::mat4 modelaux3(1.0); // para la jerarquía del holocron
246 // Holocron
247 // Centro del holocron
248 model = modelaux3;
249 color = glm::vec3(0.0f, 0.0f, 1.0f);
250 model = glm::mat4(1.0);
251 model = glm::translate(model, glm::vec3(-20.0f, 5.0f, 0.0f));
252 modelaux = model;
253 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
254 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
255 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
256 centroHolocron.RenderModel();
257
258 // Piramide 1
259 model = modelaux3;
260 model = glm::translate(model, glm::vec3(-18.05f, 6.9f, 1.95f));
261 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion10()), glm::vec3(1.0f, 1.0f, 1.0f));
262 modelaux = model;
263 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
264 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
265 color = glm::vec3(0.0f, 1.0f, 0.0f);
266 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
267 piramide1.RenderModel();
268
269 // Piramide 2
270 model = modelaux3;
271 model = glm::translate(model, glm::vec3(-18.05f, 6.95f, -1.95f));
272 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion11()), glm::vec3(-1.0f, -1.0f, 1.0f));
273 modelaux = model;
274 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
275 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
276 color = glm::vec3(1.0f, 0.0f, 0.0f);
277 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
278 piramide2.RenderModel();
279
280 // Piramide 3
281 model = modelaux3;
282 model = glm::translate(model, glm::vec3(-21.95f, 6.9f, 1.95f));
283 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion12()), glm::vec3(1.0f, -1.0f, -1.0f));
284 modelaux = model;
285 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
286 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
287 color = glm::vec3(1.0f, 0.0f, 1.0f);
288 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
289 piramide3.RenderModel();
290
291 // Piramide 4
292 model = modelaux3;
293 model = glm::translate(model, glm::vec3(-21.95f, 6.95f, -1.95f));
294 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion13()), glm::vec3(-1.0f, 1.0f, -1.0f));
295 modelaux = model;
296 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
297 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
298 color = glm::vec3(1.0f, 1.0f, 0.0f);
299 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
300 piramide4.RenderModel();
```

```

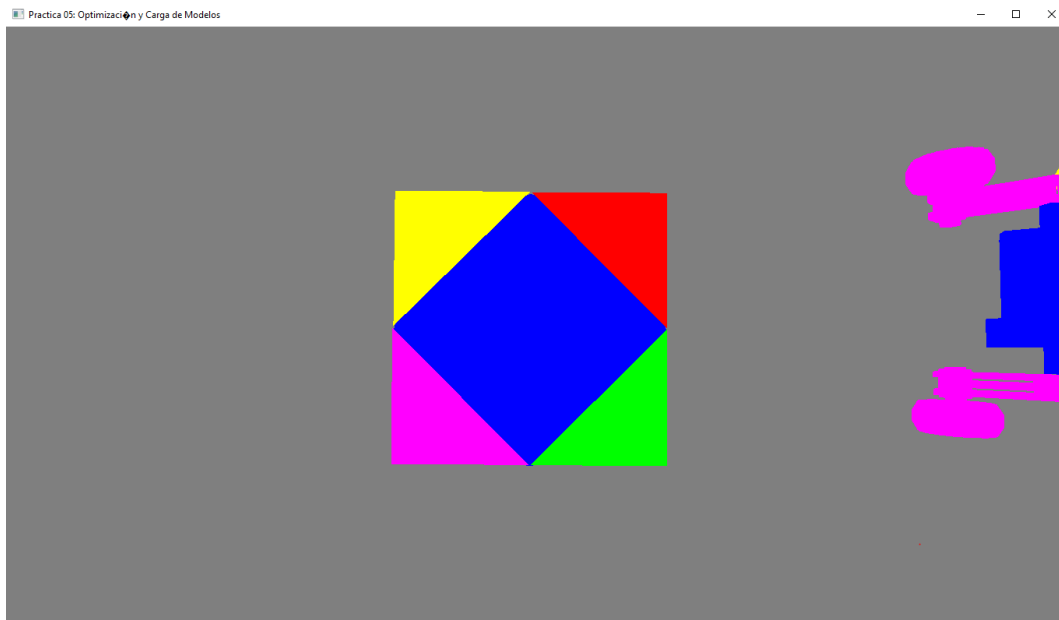
467 // Piramide 5
468 model = modelaux3;
469 model = glm::translate(model, glm::vec3(-21.95f, 3.1f, 1.95f));
470 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion14()), glm::vec3(-1.0f, -1.0f, 1.0f));
471 modelaux = model;
472 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
473 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
474 color = glm::vec3(1.0f, 0.0f, 0.0f);
475 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
476 piramide5.RenderModel();
477
478 // Piramide 6
479 model = modelaux3;
480 model = glm::translate(model, glm::vec3(-21.95f, 3.1f, -1.95f));
481 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion15()), glm::vec3(-1.0f, -1.0f, -1.0f));
482 modelaux = model;
483 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
484 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
485 color = glm::vec3(0.0f, 1.0f, 0.0f);
486 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
487 piramide6.RenderModel();
488
489 // Piramide 7
490 model = modelaux3;
491 model = glm::translate(model, glm::vec3(-18.05f, 3.1f, 1.9f));
492 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion16()), glm::vec3(-1.0f, 1.0f, -1.0f));
493 modelaux = model;
494 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
495 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
496 color = glm::vec3(1.0f, 1.0f, 0.0f);
497 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
498 piramide7.RenderModel();
499
500 // Piramide 8
501 model = modelaux3;
502 model = glm::translate(model, glm::vec3(-18.05f, 3.1f, -1.95f));
503 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion17()), glm::vec3(1.0f, -1.0f, -1.0f));
504 modelaux = model;
505 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
506 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
507 color = glm::vec3(1.0f, 0.0f, 1.0f);
508 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
509 piramide8.RenderModel();

```

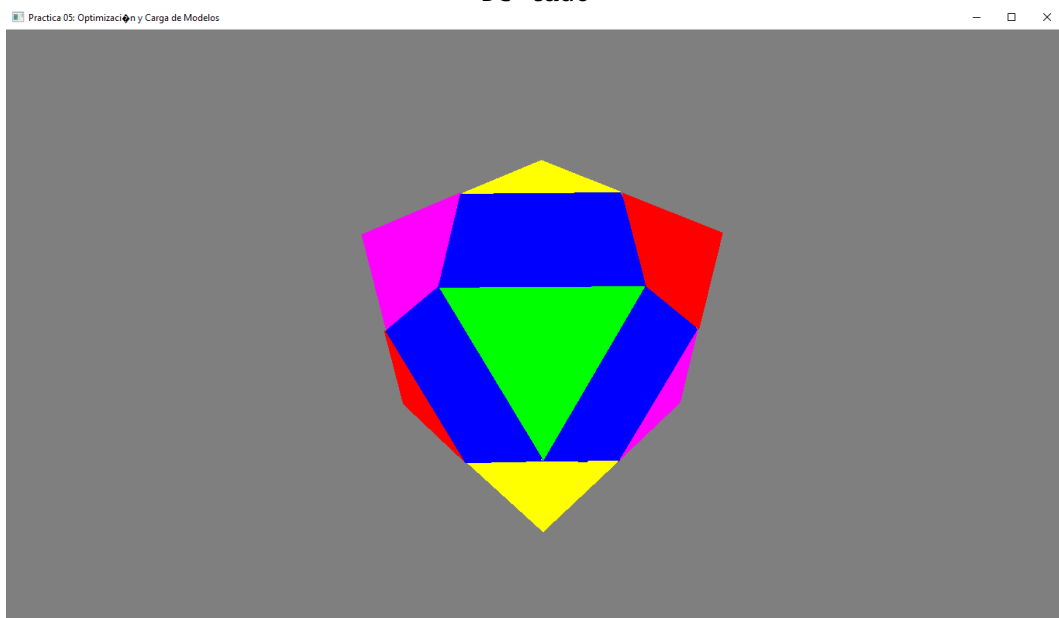
Agrego capturas de pantalla del holocron construido:



Arriba

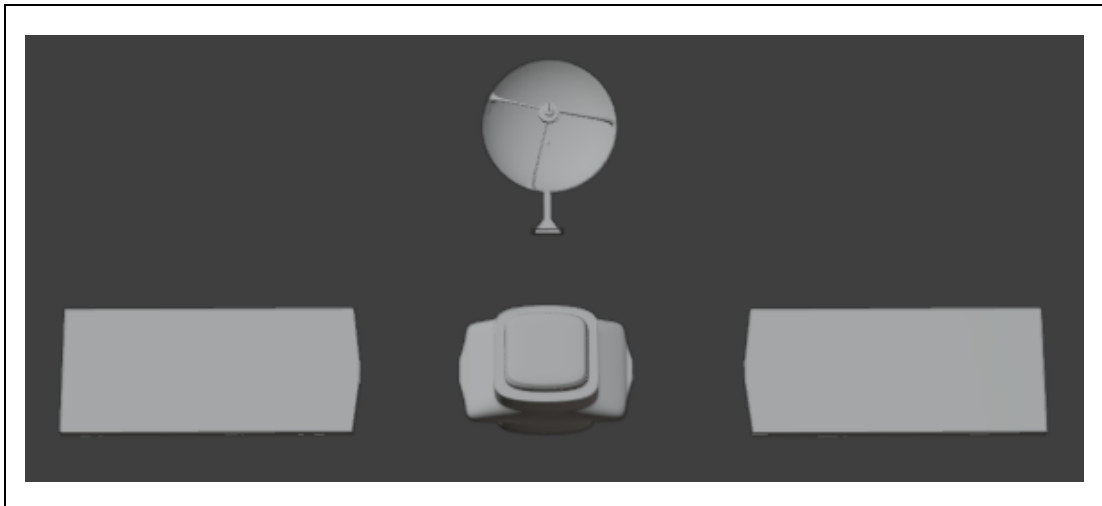


De lado



1.2 Importar el satélite con jerarquía, agregar movimiento al satélite con teclado sobre los ejes X, Y, Z y finalmente, agregar rotaciones a por lo menos 3 partes del satélite que tengan contexto adecuado. Por ejemplo, si el satélite tiene un panel de celdas solares, solo debería de girar en la conexión con el cuerpo, no alrededor de todo el cuerpo.

Primero importé y separé en *Blender* todos los elementos de interés del modelo *Satellite_320152896.fbx* (2 paneles solares, una antena y al base del satélite).



Posteriormente exporté cada elemento por separado a la carpeta *Models* y una vez hecho esto, en *P05_320152896.cpp* modifiqué los bloques de código siguientes:

- Lista de modelos a importar:

```
65 //Satelite
66 Model cuerpoSatelite;
67 Model antena;
68 Model panel1;
69 Model panel2;
```

➤ Carga de modelos:

```
214 //Satelite
215 cuerpoSatelite = Model();
216 cuerpoSatelite.LoadModel("Models/cuerpoSatelite.fbx");
217
218 antena = Model();
219 antena.LoadModel("Models/antena.fbx");
220
221 panel1 = Model();
222 panel1.LoadModel("Models/panel1.fbx");
223
224 panel2 = Model();
225 panel2.LoadModel("Models/panel2.fbx");
```

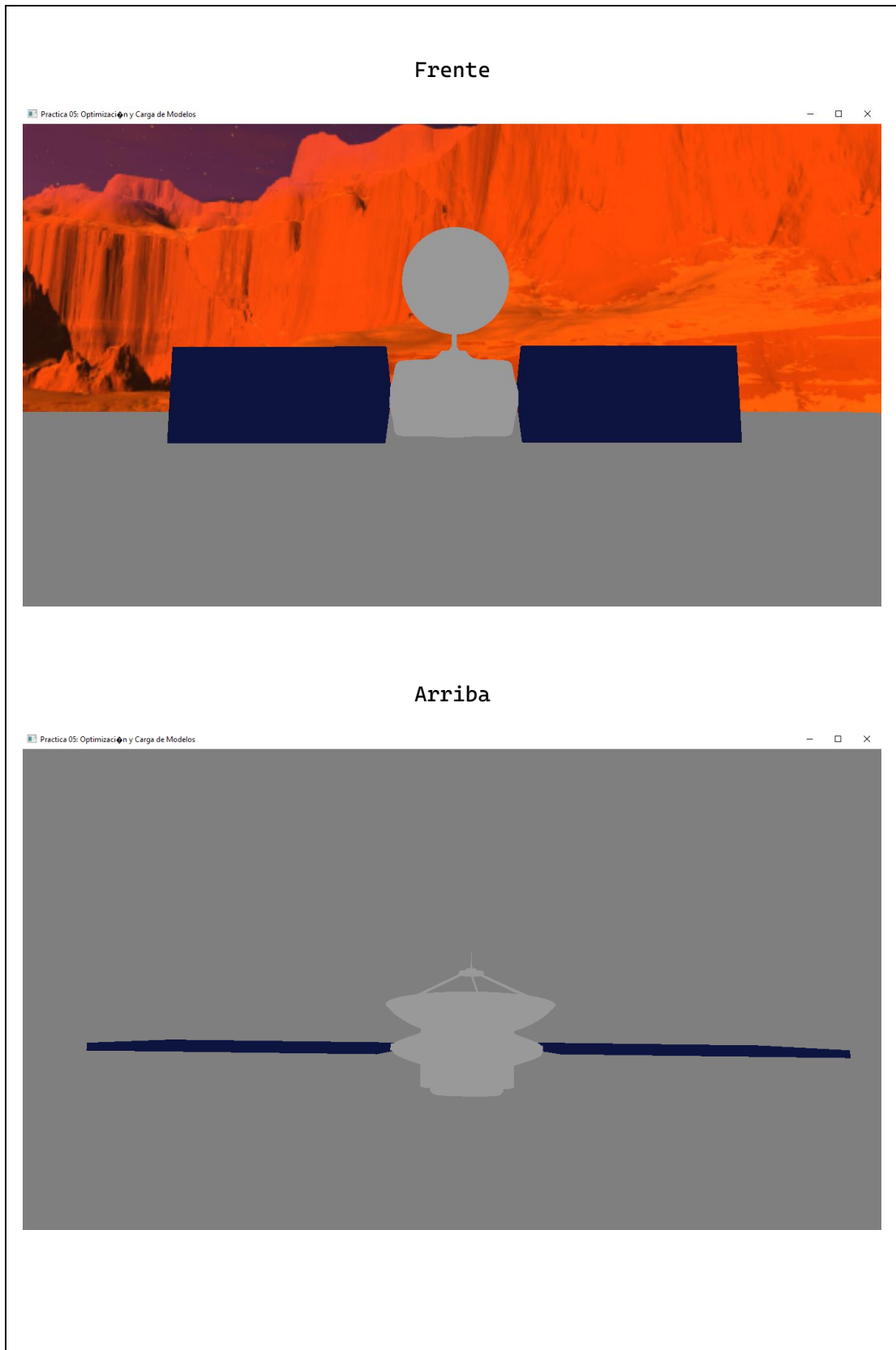
➤ Dibujo de los objetos:

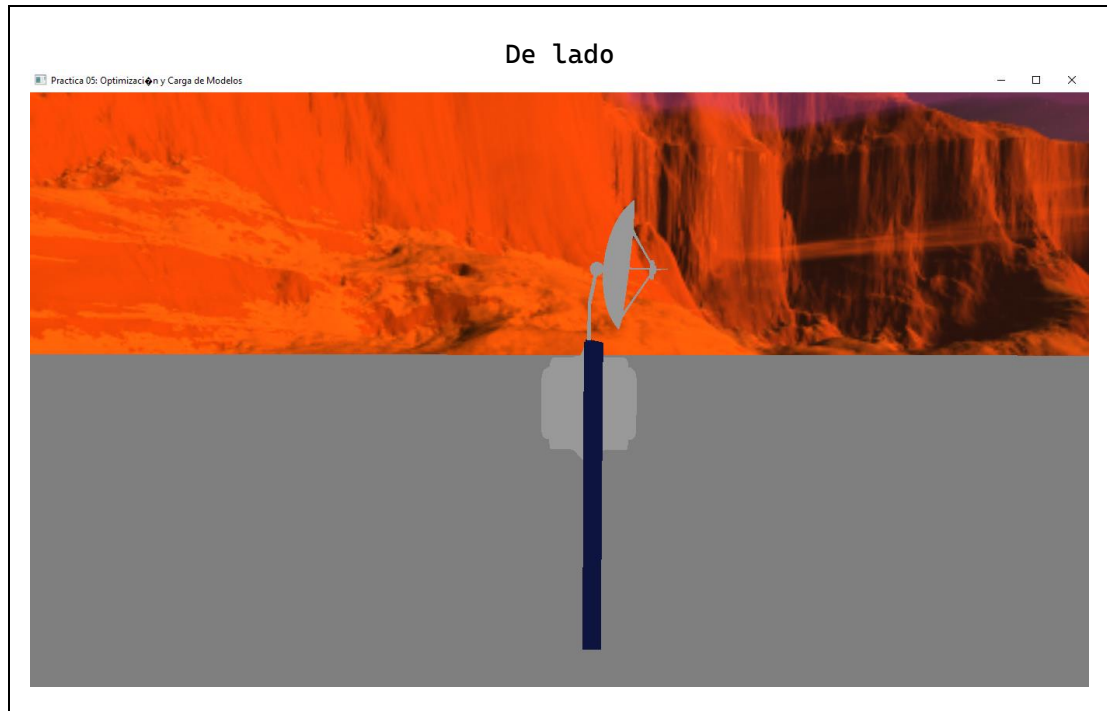
```
246 glm::mat4 modelaux4(1.0); // para la jerarquía de la base del satelite
247 glm::mat4 modelaux5(1.0); // para la jerarquía de las extensiones de la base del satélite
248
249 // Satelite
250 // Cuerpo del satelite
251 model = modelaux4;
252 color = glm::vec3(0.6f, 0.6f, 0.6f);
253 model = glm::mat4(1.0);
254 model = glm::translate(model, glm::vec3(25.0f, 5.0f, 0.0f));
255 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion18()), glm::vec3(1.0f, 0.0f, 0.0f));
256 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion19()), glm::vec3(0.0f, 1.0f, 0.0f));
257 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion20()), glm::vec3(0.0f, 0.0f, 1.0f));
258 modelaux = model;
259 model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
260 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
261 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
262 cuerpoSatelite.RenderModel();
263
264 // Antena
265 model = modelaux;
266 model = glm::translate(model, glm::vec3(0.0f, 1.0f, 0.0f));
267 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion21()), glm::vec3(0.0f, 1.0f, 0.0f));
268 modelaux5 = model;
269 model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
270 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
271 color = glm::vec3(0.6f, 0.6f, 0.6f);
272 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
273 antena.RenderModel();
274
275 // Panel solar 1
276 model = modelaux;
277 model = glm::translate(model, glm::vec3(1.7f, 0.0f, 0.0f));
278 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion22()), glm::vec3(1.0f, 0.0f, 0.0f));
279 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion23()), glm::vec3(0.0f, 1.0f, 0.0f));
280 modelaux5 = model;
281 model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
282 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
283 color = glm::vec3(0.05f, 0.08f, 0.25f);
284 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
285 panel1.RenderModel();
286
287 // Panel solar 2
288 model = modelaux;
289 model = glm::translate(model, glm::vec3(-1.7f, 0.0f, 0.0f));
290 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion24()), glm::vec3(1.0f, 0.0f, 0.0f));
291 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion25()), glm::vec3(0.0f, 1.0f, 0.0f));
292 modelaux5 = model;
293 model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
294 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
295 color = glm::vec3(0.05f, 0.08f, 0.25f);
296 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
297 panel2.RenderModel();
```

Para que los paneles solares no se encimen con el cuerpo y la antena al momento de rotarlos, reutilicé la variable booleana del ejercicio de clase para que sea FALSE cuando se alcance 85° y vuelva a ser TRUE cuando alcance -85°. Para esto modifiqué *Window.cpp*.

```
359     if (key == GLFW_KEY_2)
360     {
361         if (theWindow->limiteMaximo)
362         {
363             theWindow->articulacion23 += 5.0f;
364
365             if (theWindow->articulacion23 >= 85.0f)
366             {
367                 theWindow->articulacion23 = 85.0f;
368                 theWindow->limiteMaximo = false;
369             }
370         }
371         else
372         {
373             theWindow->articulacion23 -= 5.0f;
374
375             if (theWindow->articulacion23 <= -85.0f)
376             {
377                 theWindow->articulacion23 = -85.0f;
378                 theWindow->limiteMaximo = true;
379             }
380         }
381     }
382     if (key == GLFW_KEY_3)
383     {
384         theWindow->articulacion24 += 10.0;
385     }
386     if (key == GLFW_KEY_4)
387     {
388         if (theWindow->limiteMaximo)
389         {
390             theWindow->articulacion25 += 5.0f;
391
392             if (theWindow->articulacion25 >= 85.0f)
393             {
394                 theWindow->articulacion25 = 85.0f;
395                 theWindow->limiteMaximo = false;
396             }
397         }
398         else
399         {
400             theWindow->articulacion25 -= 5.0f;
401
402             if (theWindow->articulacion25 <= -85.0f)
403             {
404                 theWindow->articulacion25 = -85.0f;
405                 theWindow->limiteMaximo = true;
406             }
407         }
408     }
```

Agrego capturas de pantalla del satélite construido:





NOTA: Las imágenes presentadas en este documento no representan al 100% el funcionamiento del holocron y el satélite, para ello se adjunta en el repositorio de GitHub un video (P05_320152896.mp4) en el cual se visualiza de mejor manera el movimiento de cada elemento.

2. Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.

Para esta práctica no se presentaron problemas.

3. Conclusión:

- a. **Los ejercicios del reporte:** Me gustó mucho que se va a seguir con el tema “espacial” y a su vez, me entusiasma aún más la realización del proyecto final para incluir todo esto que hemos visto hasta ahora.
- b. **Comentarios generales:** Considero que fueron ejercicios al nivel de lo que se enseñó en el laboratorio y sirvió para reforzar lo relacionado a la carga de modelos.
- c. **Conclusión.** Esta práctica me sirvió mucho para saber cómo importar modelos en *Blender* para separarlos y posteriormente exportarlos en sub modelos, con los cuáles realizamos las diversas figuras (holocron y satélite) que en prácticas anteriores eran hechas con primitivas. Dicho lo anterior, me gustó mucho que ahora nos enfocamos más en la correcta

traslación, escalación y rotación de los modelos importados, qué en dibujarlos manualmente elemento por elemento. Finalmente esto me da una mejor idea de cómo incorporar muchos de los elementos de cada planeta del personaje elegido para nuestro proyecto final.

Referencias bibliográficas.

vibrant-vertex. (2021, 14 junio). *Satellite 3D Free 3D model*. cgtrader.
<https://www.cgtrader.com/free-3d-models/space/other/satellite-3d>